

Scope 扩散实现与 Stream Memory 设计对比

Digital Friction 实验 — 0507 进展汇报

2026-05-07

本次汇报回答两个问题

1. Scope 扩散现在到底怎么落到代码里？

- LLM 输出的不是扩散目标，而是 `scope_amplitude`
- 代码用固定 task-family 相似度矩阵计算 Gaussian 权重
- 扩散结果扣的是其他任务家族的 `task_self_efficacy`
- 扩散量和不可控感、scope amplitude、全局 beta 有关

2. 我们的 memory 和原生 AgentSociety memory 有什么区别？

- 原生 memory 是通用的 `status + stream + vector search`
- 我们复用原生 `stream.add/search` 接口
- 但 topic、prompt 接入点、消融开关都按 digital friction 机制重做
- memory 不替代 helplessness 主公式，而是提供 episodic evidence

Scope 在当前系统中的位置



当前实现把 **scope** 理解成“这次失败会不会被 agent 泛化到相似数字任务”。LLM 负责给一个主观泛化强度，代码负责决定这个强度如何按任务相似度扩散。

LLM 只输出 scope amplitude

Attribution prompt 的核心问题

- 谁导致了这次挫折: locus
- 这种困难是否会持续: stability
- 会不会扩散到相似数字任务:
scope_amplitude

scope amplitude 标尺

0.0 = 只限当前任务

0.5 = 扩散到相似任务

1.0 = 明显泛化到类似数字任务

重要边界

- LLM 不直接修改 self-efficacy
- LLM 不决定扩散到哪些任务
- LLM 不输出 Gaussian 参数
- LLM 的 retrieval 只是证据, 不是数值更新器

event_attribution_scope_amplitude: 0.0--1.0

代码位置: examples/digital_friction_mvp/proto/attribution_inference.py 中 event_attribution_scope_amplitude 的 prompt 定义与规则。

代码里的 Gaussian 扩散公式

一次负面事件通过三步变成跨任务扣分：

$$d(i, j) = 1 - S_{ij}$$

$$w_{ij} = \exp\left(-\frac{d(i, j)^2}{2\sigma^2}\right), \quad j \neq i$$

$$\hat{w}_{ij} = \frac{w_{ij}}{\sum_{k \neq i} w_{ik}}$$

$$A_t = \beta \cdot U_t \cdot a_t$$

$$\Delta SE_{j,t}^{spill} = -A_t \cdot \hat{w}_{ij}$$

符号	代码含义
S_{ij}	固定 task-family similarity matrix
σ	PROTO_SCOPE_SPILLOVER_SIGMA, 默认 0.45
β	PROTO_SCOPE_SPILLOVER_BETA, 默认 1.2
U_t	event_level_uncontrollability 归一化后的不可控感
a_t	LLM 输出的 event_attribution_scope_amplitude

触发条件：不是每次失败都会扩散

必须同时满足

- ① outcome 属于失败类：
- ② failure_after_attempt
- ③ failure_even_with_help
- ④ abandon_midway
- ⑤ event attribution 状态为 ok
- ⑥ scope_amplitude 不低于阈值

默认参数

- `PROTO_SCOPE_SPILLOVER_BETA=1.2`
- `PROTO_SCOPE_SPILLOVER_THRESHOLD=0.15`
- `PROTO_SCOPE_SPILLOVER_SIGMA=0.45`

如果不满足

- `scope_spillover_total=0`
- `scope_spillover_targets_json` 记录为空 JSON

代码位置：experience_memory.py 的 `_apply_scope_spillover(...)`。

固定相似度矩阵：六类数字任务

	Nav	Login	Search	Form	Service	Payment
Nav	1.000	0.635	0.692	0.620	0.746	0.603
Login	0.635	1.000	0.517	0.717	0.626	0.598
Search	0.692	0.517	1.000	0.621	0.711	0.693
Form	0.620	0.717	0.621	1.000	0.695	0.615
Service	0.746	0.626	0.711	0.695	1.000	0.643
Payment	0.603	0.598	0.693	0.615	0.643	1.000

矩阵越高，表示两个数字任务越相似。Gaussian 权重会先按 $1 - S_{ij}$ 转成距离，再归一化，所以每次扩散的总量固定为 A_t ，只是分配给不同邻居任务的比例不同。

Nav = navigation service location; Login = account login verification; Search = information search judgment; Form = profile form upload; Service = service application submission; Payment = payment risk confirmation.

实际更新对象：不是 helplessness，而是其他任务的 self-efficacy

代码更新的是

$$SE_j \leftarrow \text{clamp}(SE_j - A_t \hat{w}_{ij})$$

- 源任务本身先按 outcome 更新
- 其他 task family 再接受 spillover 扣分
- 扣分结果写回 task_domain_memory
- 后续通过 memory features 影响策略和 outcome

审计记录

- event_attribution_scope_amplitude
- scope_spillover_total
- scope_spillover_targets_json
- 这些字段写入 attempt 表和 payload

```
{"payment_risk_confirmation": 0.17,  
 "profile_form_upload": 0.14}
```


原生 AgentSociety memory: 通用状态与经历流

原生结构

- KVMemory: key-value 状态记忆
- StreamMemory: 时间顺序的经历流
- VectorStore: FastEmbed + Qdrant 语义检索
- Memory: 包装出 status 和 stream

- [topic]: description [day: ..., time: ..., location: ...]

原生 stream 接口

- 写入: `memory.stream.add(topic, description)`
- 检索: `memory.stream.search(query, topic, top_k)`
- 当天检索:
`memory.stream.search_today(...)`
- 返回格式带 topic、day、time、location

参考 GitHub 原仓库: [agentsociety/memory/memory.py](#) 与 [agentsociety/vectorstore/vectorstore.py](#)。

原生 cognition 如何用 stream memory



原生 AgentSociety 的设计更像“通用认知系统”：事件进入 stream，cognition block 从 stream 中检索经历，LLM 形成新的 thought，再把 thought 作为 cognition 写回 stream。

参考 GitHub 原仓库：cityagent/blocks/cognition_block.py 中 `memory.stream.search`、`search_today`、`stream.add(topic='cognition', ...)`。

我们的 stream memory: 复用原生接口, 但限制业务边界

我们复用的原生部分

- 仍用 `memory.stream.add`
- 仍用 `memory.stream.search`
- 仍保留原生返回格式
- 不访问私有 `_vectorstore`
- 不新建第二套 memory 系统

我们重做的实验边界

- topic 全部围绕 digital friction
- 只在指定 prompt 里读取
- retrieval 有 runtime 开关
- audit packet 写入 payload
- memory 不直接改 helplessness 主公式

Phase 0-3: 我们现在的 memory 层

Phase	做什么	topic / 接入点
Phase 0	每次数字任务结束写 episode, 但不读取	digital_task_episode, digital_help_episode, digital_recovery_episode
Phase 1	task appraisal 前检索过去 episode	prompt 中加入 Retrieved Episodic Memory
Phase 2	event attribution 前检索相似 episode	prompt 中加入 Retrieved Similar Episodes
Phase 3	daily reflection 写回 stream, stage summary 参考 episode stream	digital_daily_reflection, digital_friction_stage_summary

核心原则: raw episode 可以作为证据进入 appraisal / attribution; reflection 和 stage summary 只服务 interview 与 qualitative coherence, 不回流到 Phase 1/2, 避免双重计数。

Phase 0: 写数字任务经历

每次任务结束写

- digital_task_episode
- 标签: family、friction、outcome、strategy、help、negative、uncontrollability
- 描述是第一人称自然语言

有求助才额外写

- digital_help_episode

关键恢复才额外写

- digital_recovery_episode
- 自己成功且近期同任务多次失败
- 或 enabling help 让任务恢复成功

```
[digital_task_episode][family=...]  
I tried payment and risk confirmation  
and could not complete it.
```

Phase 1/2: 读 raw episode, 但读给不同模块

Phase 1: task appraisal retrieval

- 发生在 `resolve_task_appraisal` 之前
- query: task family + readable label + failure/success/help
- 进入 prompt 的字段:
- Retrieved Episodic Memory
- 影响的是任务前主观判断

两者都只检索 raw topics: task/help/recovery, 不检索 reflection 或 stage summary。

Phase 2: attribution retrieval

- 发生在 `infer_event_attribution` 之前
- query: task family + recovery/help/similar setback
- 进入 prompt 的字段:
- Retrieved Similar Episodes
- 影响的是失败后如何解释 scope/stability

Phase 3: reflection 更接近原生 memory, 但不改变主机制

Daily reflection 写回 stream

- topic: digital_daily_reflection
- 来源: 已有
maybe_generate_daily_reflection
- 格式: 稳定标签 + 第一人称 reflection
- 只记录, 不进入 appraisal / attribution

Stage summary 参考 raw episodes

- 写入 topic 仍是
digital_friction_stage_summary
- 写入前 search raw task/help/recovery episodes
- description 中加入短的
episode_evidence
- stage interview 读取 stage summary

我们与原生 AgentSociety memory 的主要区别

维度	原生 AgentSociety	我们的 digital friction memory
目标	通用 cityagent cognition: 记录事件、回顾当天、更新 thought	实验机制层: 记录数字摩擦经历, 给 appraisal / attribution 提供可审计证据
状态层	KVMemory 保存通用 agent 状态	另有结构化状态: task domain memory、help effect memory、emotion、helplessness
Stream topic	开放式 topic, 例如 cognition 和日常事件	受控 topic: task/help/recovery/reflection/stage summary
检索使用	cognition block 可开放使用 stream 与 search_today	Phase 1/2 只读 raw episode topics, Phase 3 只服务 reflection/interview
更新主机制	原生 thought 可成为 cognition 状态的一部分	stream 不直接改 helplessness 公式; scope 扩散仍由结构化 attribution + Gaussian 代码完成
消融审计	原生系统不以实验消融为核心	每个阶段有开关、condition、hash、count、payload audit

为什么这样设计

向原生 AgentSociety 学习

- 用 stream 保存经历流
- 用 topic 做记忆分类
- 用 semantic search 回忆相关经历
- 用 reflection 把经历组织成叙事

但不照搬原生 cognition

- 原生 cityagent 是城市生活认知
- 我们研究的是 digital friction 与 helplessness
- 所以必须保留机制边界和可消融性
- 避免 stream 变成第二套不可控状态更新器

复用 memory 接口 $\neq$ 复制原生业务逻辑

当前实现的可验证信号

Scope 扩散是否生效

- attempt 表中
event_attribution_scope_amplitude
- scope_spillover_total > 0
- scope_spillover_targets_json 非空
- 相邻 task family 的 self-efficacy 有下降

Stream memory 是否生效

- payload 中有
stream_episode_recording
- Phase 1/2 audit 中 status 为 ok 或 empty
- count 大于 0 表示检索到了历史 episode
- stream 中存在 daily reflection / stage summary topic

总结

- Scope 扩散现在是“LLM 判断主观泛化强度 + 本地 Gaussian 相似度扩散”的混合实现
- 扩散不是直接加 helplessness, 而是扣其他任务家族的 `task_self_efficacy`
- 我们的 memory 复用了 AgentSociety 原生 stream 接口和 semantic search 思路
- 但我们的 topic、retrieval 边界、audit 和消融开关都服务于 digital friction 机制
- Phase 3 让 reflection 更接近原生 memory 的“经历流 + 反思写回”，但仍不进入主公式，避免双重计数

机制主线保持结构化，memory 作为可检索的经历证据层